

UPPSALA UNIVERSITY



PARALLELL OCH DISTRIBUTERAD PROGRAMMERING

1TD070

Assignment 3: The Parallel Quick Sort Algorithm

Alva Christensson, Elin Djurberg and Marcus Moritz

May 12, 2026

Contents

1	Problem description	2
1.1	The Parallel Quick-sort algorithm	2
2	Implementation	2
3	Numerical Experiments and Performance	3
4	Discussion	6
5	References	7

1 Problem description

The task is to implement a parallel quick-sort algorithm using C and MPI. As a guide for the implementation two header files including the functions to be implemented were included in the instructions. The implementation should follow the algorithm described below in Section 1.1, this algorithm was also given in the instructions for the assignment [1]. In this algorithm three different pivot strategies should be implemented. These three are as said in the instructions [1]:

1. Select the median in one processor in each group of processors.
2. Select the median of all medians in each processor group.
3. Select the mean value of all medians in each processor group.

1.1 The Parallel Quick-sort algorithm

1. Divide the data into p equal parts, one per process
2. Sort the data locally for each process
3. Perform global sort
 - (a) Select pivot element within each process set
 - (b) Locally in each process, divide the data into two sets according to the pivot (smaller or larger)
 - (c) Split the processes into two groups and exchange data pairwise between them so that all processes in one group get data less than the pivot and the others get data larger than the pivot.
 - (d) Merge the two sets of numbers in each process into one sorted list
4. Repeat 3.1- 3.4 recursively for each half until each group consists of one single process.

2 Implementation

Our main function starts with reading in all the arguments and saving them. We then initialize MPI and get the size and rank. Only the root process reads the input data from the file and then it distributes it by the function `distribute_from_root()`. We had to use `MPI_Scatterv()` in `distribute_from_root()` as all the process will have different size of lists to sort[2]. We then perform the `qsort()` function from C locally on each process [3]. After that we call `global_sort()` which follows the algorithm described in Section 1.1. After `global_sort()` is done, all the sorted data is gathered using the function `gather_on_root`. This function uses the MPI function `MPI_Gatherv()` which

makes it possible to gather arrays of different length on the root [4]. Then the function `check_and_print()` which outputs the data in a txt file is used. The check and print function uses the function `compare()` which was taken from the qsort algorithm from `geeks_for_geeks` [3]. To time the program we used the built in `MPI_Wtime()` on each process and choose the slowest one with the help of `MPI_Reduce()`. We ran the program 5 times and took the average of the run times as the final result.

The `global_sort()` function is a recursive function which starts by checking if the final step where the group consists of only one process has been reached. If it has it just returns the length that is an input to the function. If not an recursive step is commenced. This starts by calling the `select_pivot()` function with the selected pivot strategy, the pivot strategy is selected by choosing a number when running the code and this function will return the first index after the pivot element. Two arrays are allocated and the elements of the original list up until and including the pivot element is added to array `v1` and the rest of the elements are added to `v2`. The goal now is to put all the `v1` array in processes ($< \frac{num_processes}{2}$) and all the `v2` arrays in processes ($\geq \frac{num_processes}{2}$). This is done by first sending and receiving the length of the arrays in order to allocate the correct amount of space for the arrays. After the new array is allocated the arrays are exchanged with process $\frac{size}{2+rank}$ if the process is sending `v2` and exchanged with process $\frac{rank-size}{2}$ if the process is sending `v1`. This communication was done using the example of deadlock free communication from Lecture 3 slide 30 [5]. These arrays are then merged using the function `merge_ascending()` which merges and sorts the two arrays to a new array. After this the processes are split into two groups using the function `MPI_Comm_split` and then the entire function is ran again recursively on the smaller groups of processes [6].

3 Numerical Experiments and Performance

As already mentioned in Section 2 the list is checked using the implemented `check_and_print()` function. How the time for each run is selected is also explained in Section 2. When conducting the numerical and performance experiments five runs were done for each number of processes and the median time for these were calculated and shown in Table 1.

The speedup was calculated using Equation 1. Here T_s is the serial time and T_p is the parallel time. The speedup for strong scalability can be seen in Table 4 and for weak scalability in Table 5. Figure X shows the strong scalability speedup plotted against the ideal speedup. This speedup was plotted for the problem size X. The weak scalability, as can be seen in Figure X compared to the ideal speedup, was plotted by varying the input size, starting with the smallest to the largest while at the same time increasing the number of processors. The code for plotting the speed-up for strong and weak scalability can be found in the submission.

Table 1: Timing table for pivot strategy 1

$p \setminus n (10^6)$	125	250	500	1000
1	18.5	37.7	78.9	326
2	9.39	19.2	40.1	163
4	4.80	9.71	20.2	83.8
8	2.49	5.06	22.9	41.1
16	1.41	3.20	13.3	14.8
32	0.80	2.30	4.72	7.72

Table 2: Timing table for pivot strategy 2

$p \setminus n (10^6)$	125	250	500	1000
1	18.4	38.1	79.1	329
2	9.36	19.4	39.8	165
4	4.77	9.82	20.4	83.9
8	2.49	5.12	24.9	23.9
16	1.42	5.90	12.2	12.4
32	0.82	3.63	5.00	7.70

Table 3: Timing table for pivot strategy 3

$p \setminus n (10^6)$	125	250	500	1000
1	18.5	38.2	79.6	326
2	9.24	19.3	40.1	165
4	4.78	9.79	20.3	89.5
8	2.50	5.03	25.0	25.3
16	1.38	6.18	14.3	18.5
32	0.82	3.77	5.44	8.15

$$S = \frac{T_s}{T_p} \quad (1)$$

For strong scalability we used our biggest problem size ($2000 \cdot 10^6$) as it limits the impact of any potential overhead and increased the number of process used up to 32 process. We choose to exclude 48 as it is not a power of 2 and we had issues making it work. To find the weak scalability we moved along the

Table 4: Speed-up for strong scalability for all 3 different pivot strategies (PS)

$p \backslash$ PS	1	2	3
1	1.00	1.00	1.00
2	2.00	1.99	1.98
4	3.89	3.92	3.64
8	7.93	13.77	12.89
16	22.03	26.53	17.62
32	42.23	42.73	40.00

Table 5: Speed-up for weak scalability for all 3 different pivot strategies (PS)

$p \backslash$ PS	1	2	3
1	1.0000	1.0000	1.0000
2	0.9635	0.9485	0.9585
4	0.9505	0.9510	0.9507
8	0.4915	0.8536	0.8024

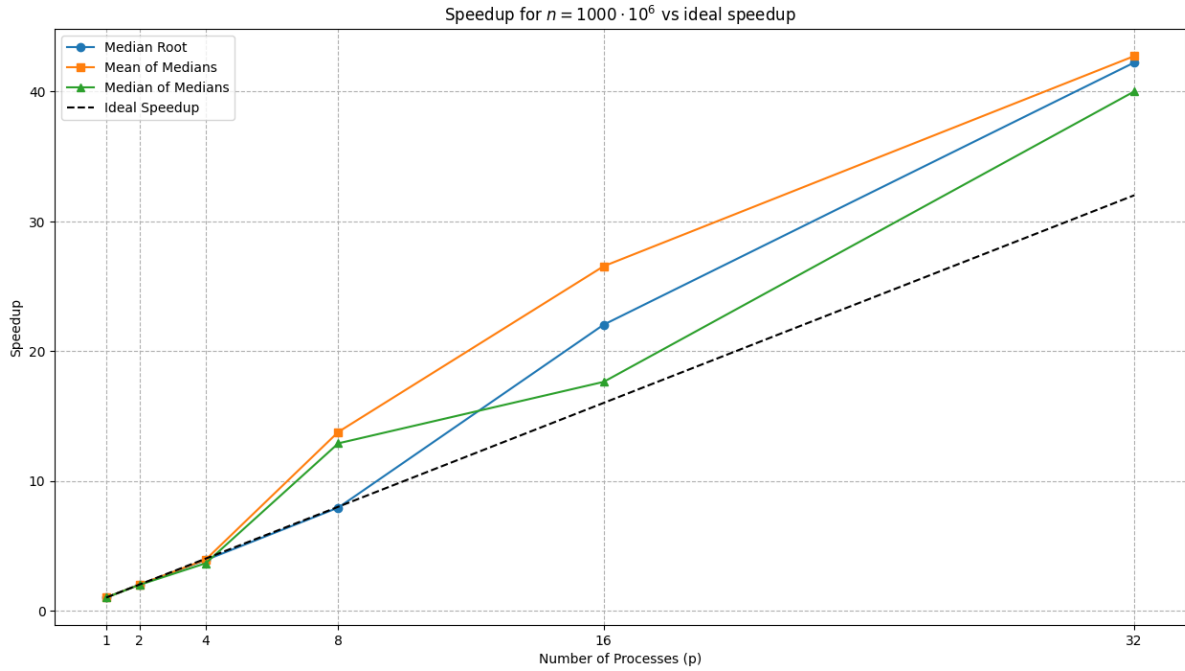


Figure 1: Speed-up for strong scalability.

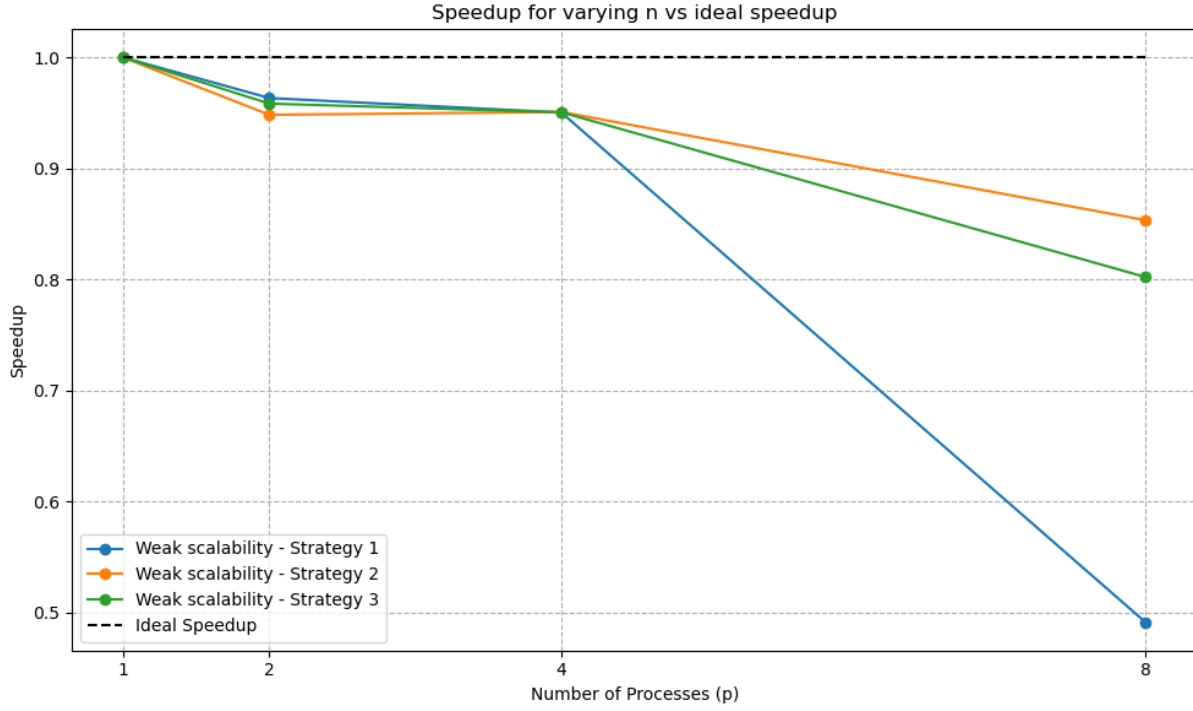


Figure 2: Speed-up for weak scalability.

4 Discussion

We can see in Table 4 that our results look as expected up to 4 process but after that only pivot strategy 1 keeps below the ideal speedup for 8 process and after that all pivot strategies show some really weird results. Something is wrong in our code but we can not find the reason for it. We can also see in the timings tables for pivot strategy 1, Table 1, pivot strategy 2, Table 2 and in pivot strategy 3, Table 3, we get a slower timing when going from 4 to 8 process with a problem size of $500 \cdot 10^6$. This is also really weird. We would really appreciate a new set of opinions on our code.

5 References

- [1] R. Iakymchuk, “Assignment no.3,” *Studium*, pp. 1–4, 2026.
- [2] T. O. project, *Mpi_scatterv(3) man page (version 1.4.5)*, Accessed: 2026-05-08, 2026. [Online]. Available: https://www.open-mpi.org/doc/v1.4/man3/MPI_Scatterv.3.php?fbclid=IwY2xjawRqbYVleHRuA2FlbQlXMABicmlkETBON2hLMExYQlBwbXBzN3Flc3J0YwZhchBfaWlXjVtEufvxC2rAAAt2t0gpsEuTPxr02PB1muLprw3Ksjof-_aem_IemZNIst5Vb9J60gk57MOQ.
- [3] GeeksforGeeks, *Qsort() function in c*, Accessed: 2026-05-04, 2025. [Online]. Available: <https://www.geeksforgeeks.org/c/qsort-function-in-c/>.
- [4] RookieHPC, *Mpi_gatherv*, Accessed: 2026-05-08. [Online]. Available: https://rookiehpc.org/mpi/docs/mpi_gatherv/index.html.
- [5] R. Iakymchuk, “Message passing interface,” *Studium*, pp. 1–4, 2026.
- [6] RookieHPC, *Mpi_comm_split*, Accessed: 2026-05-08. [Online]. Available: https://rookiehpc.org/mpi/docs/mpi_comm_split/index.html.